

OpenClaw Workflow

29 May 2026 12:00

Build Production-Grade Agentic Workflow Automation Platform Using OpenClaw

You are a Principal AI Architect, Senior Full-Stack Engineer, and Platform Engineer.

Build from scratch a production-quality Agentic Workflow Automation Platform using OpenClaw as the underlying agent runtime and tool execution framework.

Existing Assets

Refer to the folder:

C:\allCode\code_DK\openclaw\agents_tools_library\library\tools

This folder contains multiple tools.

Each tool may contain:

tool.py

README.md

main.py

Requirements:

1. Automatically discover all tools from this folder.
2. Parse README.md to understand tool capabilities.
3. Load tool metadata dynamically.
4. Register tools into OpenClaw.
5. Make tools available to agents at runtime.
6. Build a generic tool registry framework.
7. Support adding new tools without code changes.

Objective

Build a complete production-grade AI Agentic Workflow Automation Platform similar to:

- n8n
- LangFlow
- Dify Workflow
- CrewAI Studio
- Copilot Studio

but powered by OpenClaw agents.

The platform should allow users to:

- Create workflows visually
- Configure agents
- Connect tools
- Define triggers
- Execute workflows
- Monitor executions
- Approve agent decisions
- View logs
- Manage agents
- Manage tool libraries

Technology Stack

Frontend

React

Requirements:

- React 19+
- Typescript
- Tailwind
- React Flow
- Zustand
- Axios
- Material UI

Backend

Python

Framework:

- FastAPI

Requirements:

- FastAPI
- Pydantic
- Uvicorn
- AsyncIO

Agent Runtime

OpenClaw

Storage

JSON files only

Do not use:

- PostgreSQL
- MongoDB
- MySQL
- Redis

Use structured JSON storage for:

agents.json

workflows.json

executions.json

tool_registry.json

users.json

audit_logs.json

settings.json

Implement storage abstraction layer so database can be introduced later.

Interfaces

Interface 1

AI Chatbot

Capabilities:

- Natural language workflow creation
- Workflow execution
- Agent management
- Tool discovery
- Workflow debugging

Example:

"Create a workflow that reads email attachments and stores them in SharePoint"

System should generate workflow automatically.

Interface 2

WhatsApp

Users should be able to:

- Start workflows
- Query workflow status
- Receive approvals
- Receive alerts
- Interact with agents

Implement via WhatsApp Business API abstraction layer.

Design such that Twilio or Meta APIs can be plugged in later.

Core Modules

1. Workflow Designer

Build visual workflow builder.

Support drag-drop nodes.

Node types:

Trigger Nodes

- HTTP Trigger
- Cron Trigger
- Manual Trigger
- Webhook Trigger
- Email Trigger
- WhatsApp Trigger
- File Upload Trigger
- Google Sheet Row Trigger

Agent Nodes

- OpenClaw Agent
- Supervisor Agent
- Planner Agent
- Research Agent
- Executor Agent
- Reviewer Agent

Logic Nodes

- If Else
- Switch
- Parallel
- Merge
- Loop
- Wait
- Approval

Tool Nodes

Dynamically generated from discovered tools.

Action Nodes

- Send Email
- Send WhatsApp
- API Call
- File Write

- Generate Report

2. Workflow Engine

Implement execution engine.

Requirements:

- DAG execution
- Sequential execution
- Parallel execution
- Retry policies
- Timeout policies
- Error handling
- Compensation logic
- Checkpointing

Execution states:

Pending

Running

Paused

Waiting Approval

Completed

Failed

Cancelled

3. OpenClaw Agent Layer

Create Agent Runtime Manager.

Features:

Agent Registry

Store:

```
{  
  "agent_id": "",  
  "name": "",  
  "description": "",  
  "role": "",  
  "tools": []  
}
```

Agent Types

- Supervisor
- Planner
- Executor
- Researcher
- Reviewer
- Custom Agent

Capabilities

- Tool calling
- Memory
- Multi-agent collaboration
- Delegation
- Reflection
- Planning

4. Tool Registry Framework

Automatically scan:

library/tools

For every tool:

- Discover metadata
- Generate schema
- Register capability
- Create OpenClaw wrapper

Expose APIs:

GET /tools

GET /tools/{id}

POST /tools/execute

5. Workflow AI Builder

Create AI Workflow Generation Agent.

User provides:

Build a workflow that monitors emails and stores invoices.

System should:

Step 1

Understand intent.

Step 2

Discover suitable tools.

Step 3

Create workflow graph.

Step 4

Generate workflow JSON.

Step 5

Render automatically in UI.

6. Human In The Loop

Approval node.

Support:

- Email approvals
- WhatsApp approvals
- UI approvals

Actions:

Approve

Reject

Request Changes

Escalate

7. Execution Monitoring

Dashboard:

- Running workflows
- Completed workflows
- Failed workflows
- Queue depth
- Active agents

- Tool usage
- Execution timeline.

8. Audit Logging

Track:

- Agent actions
- Tool calls
- User actions
- Workflow executions

Store:

```
{  
  "timestamp":"","  
  "workflow":"","  
  "agent":"","  
  "action":"","  
  "result":"","  
}
```

9. Security

Implement:

Authentication

JWT

Authorization

RBAC

Roles:

- Admin
- Designer
- Operator
- Viewer

Agent Security

- Tool allow list
- Execution limits
- Timeout controls
- Sandboxed execution

10. WhatsApp Integration Layer

Create abstraction:

class MessagingProvider:

Implement:

WhatsAppProvider

TwilioProvider

Future providers:

TelegramProvider

TeamsProvider

SlackProvider

Backend APIs

Provide complete APIs.

Examples:

POST /workflow/create
POST /workflow/run
GET /workflow/status/{id}
POST /agent/create
GET /agent/list
GET /tools
POST /tools/execute
POST /approval/respond
Generate complete OpenAPI specification.

Frontend Features

Pages:

Dashboard

Workflow Builder

Workflow Executions

Agent Manager

Tool Library

Approvals

Audit Logs

Settings

Chatbot

Use React Flow for workflow canvas.

Deliverables

Generate:

1

Complete folder structure.

2

Backend architecture.

3

Frontend architecture.

4

Workflow JSON schema.

5

Agent schema.

6

Tool registry framework.

7

Execution engine implementation.

8

OpenClaw integration layer.

9

WhatsApp integration framework.

10

AI Workflow Generator.

11

Complete source code.

12

README.

13

Deployment guide.

14

Sample workflows.

15

Unit tests.

16

Integration tests.

Coding Standards

Requirements:

- Production quality
- Modular architecture
- SOLID principles
- Clean Architecture
- Dependency Injection
- Async first
- Type safety
- Comprehensive logging
- Error handling
- Configuration driven

Generate code incrementally.

Start by analyzing the tool library and designing the complete architecture before implementation.

Produce architecture diagrams, folder structures, schemas, and implementation roadmap before writing code.